Javascript

AULA 03

PROFESSOR MAROMO

Javascript

Estruturas de Controle

Como conceitos básicos veremos nesta aula: estruturas de controle de decisão e de repetição

Agenda

Conceitos

_ Operadores

_ Estruturas de Decisão

_ Estruturas de Repetição

Operadores

Em programação, os operadores são símbolos ou palavras-chave especiais que realizam operações em um ou mais valores, produzindo um resultado. Os operadores são fundamentais para executar cálculos, tomar decisões, comparar valores e realizar diversas outras manipulações nos dados dentro de um programa.

Operadores

Os operadores podem ser classificados em diferentes categorias, dependendo do tipo de operação que executam. Alguns dos principais tipos de operadores incluem:

Tipos de Operadores

1. Operadores Aritméticos: Esses operadores são utilizados para realizar operações matemáticas básicas, como adição, subtração, multiplicação, divisão e cálculos de resto. Eles permitem manipular números e obter resultados com base em fórmulas e expressões matemáticas.

javascript

```javascript
let a = 5;
let b = 2;

console.log(a + b);  // Resultado: 7 (adição)
console.log(a - b);  // Resultado: 3 (subtração)
console.log(a * b);  // Resultado: 10 (multiplicação)
console.log(a / b);  // Resultado: 2.5 (divisão)
console.log(a % b);  // Resultado: 1 (módulo)
console.log(a++);    // Resultado: 5 (incremento)
console.log(b--);    // Resultado: 2 (decremento)
```

Tipos de Operadores

2. Operadores de Atribuição: Esses operadores são usados para atribuir valores a variáveis. Eles permitem atualizar o valor de uma variável com base em operações aritméticas ou outros valores.

javascript

```javascript
let x = 10;

x += 5;  // Equivalente a x = x + 5
console.log(x);  // Resultado: 15

let y = 8;

y -= 3;  // Equivalente a y = y - 3
console.log(y);  // Resultado: 5
```

Tipos de Operadores

3. Operadores Relacionais: Também conhecidos como operadores de comparação, eles são usados para comparar valores e verificar relações entre eles. Esses operadores retornam um valor booleano (verdadeiro ou falso) com base na comparação realizada. Exemplos incluem igualdade, desigualdade, maior que, menor que, entre outros.

javascript

```javascript
let a = 5;
let b = 3;
let c = "5";

console.log(a == b);    // Resultado: false (igualdade)
console.log(a != b);    // Resultado: true (desigualdade)
console.log(a === c);   // Resultado: false (igualdade estrita)
console.log(a !== c);   // Resultado: true (desigualdade estrita)
console.log(a > b);     // Resultado: true (maior que)
console.log(a < b);     // Resultado: false (menor que)
console.log(a >= b);    // Resultado: true (maior ou igual a)
console.log(a <= b);    // Resultado: false (menor ou igual a)
```

Tipos de Operadores

4. Operadores Lógicos: Esses operadores são utilizados para combinar expressões lógicas e realizar operações booleanas. Eles permitem tomar decisões com base em múltiplas condições, avaliar a veracidade de expressões complexas e criar lógica condicional em programas.

javascript

```javascript
let a = 5;
let b = 3;

console.log(a > 0 && b > 0);   // Resultado: true (E lógico)
console.log(a > 0 || b > 0);   // Resultado: true (OU lógico)
console.log(!(a > 0));         // Resultado: false (NÃO lógico)
```

Tipos de Operadores

5. Operadores Bit a Bit: Esses operadores realizam operações em nível de bits em valores numéricos. Eles manipulam os bits individuais dos valores, permitindo ações como operações de máscara de bits, deslocamento de bits e operações de bit a bit.

javascript

```javascript
let a = 5;   // Representado em binário como 101
let b = 3;   // Representado em binário como 011

console.log(a & b);   // Resultado: 1 (AND bit a bit)
console.log(a | b);   // Resultado: 7 (OR bit a bit)
```

Nota

O operador AND bit a bit compara os bits correspondentes de dois valores e retorna um novo valor com os bits setados para 1 somente quando os bits correspondentes em ambos os valores são 1. Caso contrário, o bit resultante será 0. Aqui está uma tabela que ilustra o funcionamento do operador AND bit a bit:

```less
  5 (em binário: 101)
& 3 (em binário: 011)
  --------------
  1 (em binário: 001)
```

Nota 2

O operador OR bit a bit compara os bits correspondentes de dois valores e retorna um novo valor com os bits setados para 1 sempre que pelo menos um dos bits correspondentes em ambos os valores for 1. Se ambos os bits correspondentes forem 0, o bit resultante será 0. Veja o exemplo abaixo:

```less
    5 (em binário: 101)
|   3 (em binário: 011)
    -------------
    7 (em binário: 111)
```

Estruturas de decisão

- As estruturas de controle de decisão permitem que você tome decisões em seu código com base em condições específicas.
- Elas são usadas para executar diferentes blocos de código com base em testes lógicos.
- As principais estruturas de controle de decisão em JavaScript são **if**, **else if** e **else**.

if

O **if** permite executar um bloco de código se uma condição for avaliada como verdadeira. Veja um exemplo:

Exemplo if

javascript

```javascript
let idade = 18;
if (idade >= 18) {
  console.log("Você é maior de idade.");
}
```

else if:

O **else if** permite testar condições adicionais se a condição anterior não for verdadeira. Veja um exemplo:

Exemplo else if:

javascript

```javascript
let idade = 15;
if (idade >= 18) {
  console.log("Você é maior de idade.");
} else if (idade >= 13) {
  console.log("Você é um adolescente.");
} else {
  console.log("Você é uma criança.");
}
```

else:

O **else** é usado como uma opção final em um bloco de if para executar um código caso nenhuma das condições anteriores seja verdadeira. Veja um exemplo:

Exemplo else:

javascript

```javascript
let hora = 14;
if (hora < 12) {
  console.log("Bom dia!");
} else if (hora < 18) {
  console.log("Boa tarde!");
} else {
  console.log("Boa noite!");
}
```

Estruturas de Repetição

As estruturas de controle de repetição permitem que você execute um bloco de código várias vezes com base em uma condição específica. Elas são usadas quando você precisa repetir uma ação ou processar uma sequência de dados. As principais estruturas de controle de repetição em JavaScript são **while**, do...while e **for**.

while:

O while executa um bloco de código enquanto uma condição específica for verdadeira. Veja um exemplo:

Exemplo while:

javascript

```javascript
let contador = 1;
while (contador <= 5) {
  console.log(contador);
  contador++;
}
```

do...while:

O **do...while** executa um bloco de código pelo menos uma vez e continua a repetição enquanto uma condição específica for verdadeira. Veja um exemplo:

Exemplo do..while:

javascript

```javascript
let contador = 1;
do {
  console.log(contador);
  contador++;
} while (contador <= 5);
```

for:

O **for** executa um bloco de código por um número específico de vezes. Ele consiste em três partes: inicialização, condição e atualização. Veja um exemplo:

Exemplo for:

javascript

```javascript
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Nota

Uma observação importante sobre as estruturas de controle é a necessidade de ter cuidado com a criação de loops infinitos. Em loops infinitos, o código fica preso em um ciclo contínuo e não consegue sair dele, o que pode causar travamento do programa.

Para evitar loops infinitos, é fundamental garantir que a condição de parada seja alcançada em algum momento. Certifique-se de que a condição do loop seja atualizada corretamente em cada iteração para evitar que o loop seja executado indefinidamente.

Exercícios para prática

Decisão

1. Verificar se um número é par ou ímpar.
2. Verificar se uma pessoa é adulta (idade maior ou igual a 18) e se possui carteira de motorista.
3. Verificar se uma palavra é um palíndromo (se pode ser lida da mesma forma de trás para frente).
4. Verificar se um ano é bissexto.
5. Verificar se um número é positivo, negativo ou zero.
6. Classificar um estudante com base em sua pontuação em uma prova (A, B, C, D ou F).
7. Verificar se uma senha digitada pelo usuário é válida (deve ter pelo menos 8 caracteres e conter números e letras).
8. Verificar se um número é divisível por outro número.
9. Verificar se um triângulo é equilátero, isósceles ou escaleno com base em seus lados.
10. Verificar se um número é primo.

Exercícios para prática

Repetição

1. Imprimir os números de 1 a 10.
2. Calcular a soma dos números de 1 a 100.
3. Imprimir a tabuada de multiplicação de um número específico.
4. Imprimir os números pares de 1 a 50.
5. Calcular o fatorial de um número.
6. Imprimir os elementos de um array.
7. Calcular a média de uma lista de números.
8. Imprimir os caracteres de uma string de trás para frente.
9. Gerar uma sequência de Fibonacci até um número específico.
10. Encontrar o maior número em um array.

Conclusão

As estruturas de controle são elementos essenciais da programação que nos permitem tomar decisões e controlar o fluxo do nosso código. Ao entender e dominar as estruturas de controle, você se torna capaz de criar programas mais sofisticados, flexíveis e poderosos.

Ao utilizar essas estruturas de controle, é fundamental garantir a clareza e a legibilidade do código. Isso inclui nomear variáveis de forma significativa, indentar corretamente o código, adicionar comentários explicativos e seguir boas práticas de programação. Dessa forma, facilitamos a compreensão do código e evitamos erros ou comportamentos inesperados.

FIM

Que a vida seja como uma aventura emocionante, com desafios que nos tornam mais fortes a cada passo.